1 Pipeline and architecture diagrams

This section sketches the control- and data-flow structures that the prose chapters describe algebraically: the per-epoch estimation loop (Figure 1), the shared geometry kernel behind the three differencing regimes (Figure 2), the real-time SSR correction pipeline (Figure 3), the real-time streaming front-end that feeds it (Figure 4), and the PPP-RTK network producer's joint estimator (Figure 5). Boxes name the function or class that realises each step, so the diagrams double as a map into `src/`.

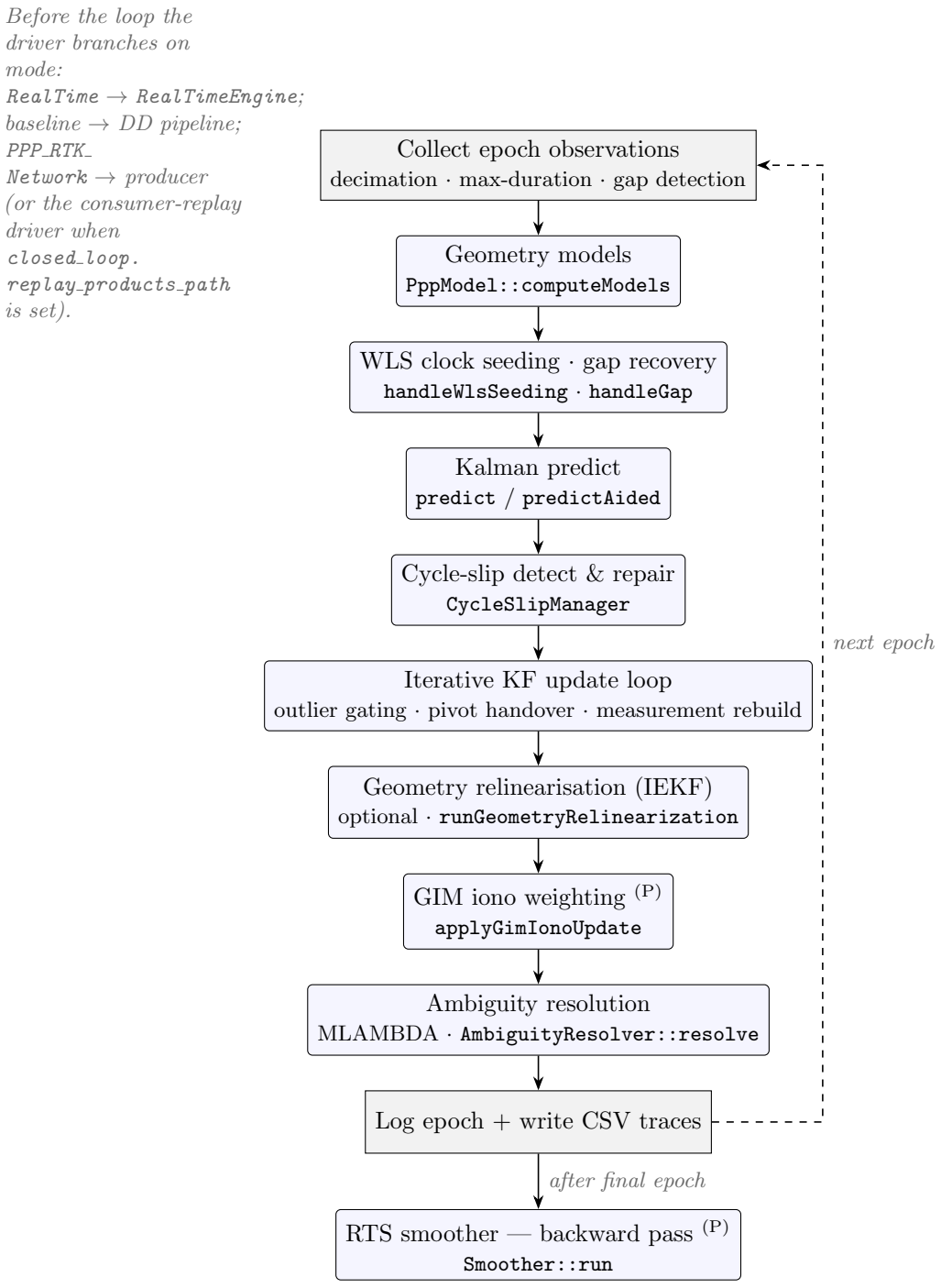


Figure 1: Per-epoch post-processing loop in `Application::run`. Solid arrows are the intra-epoch sequence; the dashed arrow is the epoch-to-epoch feedback. Steps tagged (P) run in post-processing only and are skipped on the real-time path (`RealTimeEngine`). The smoother runs once after the forward pass completes.

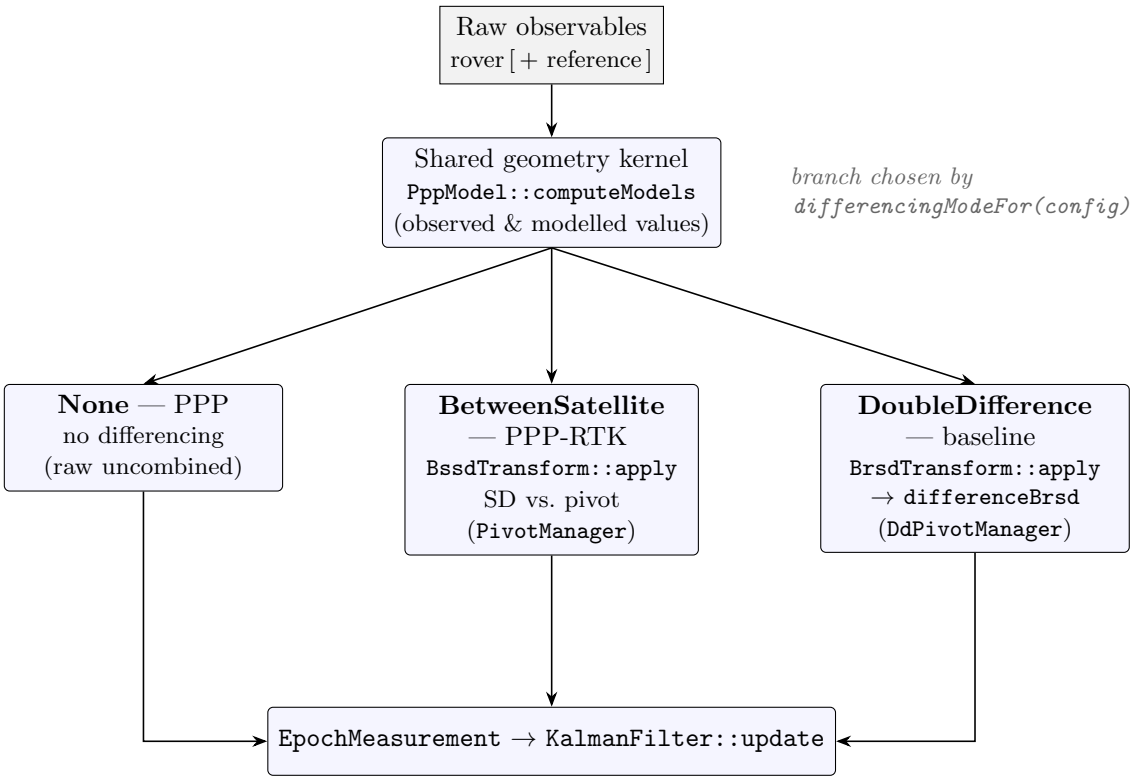


Figure 2: The three differencing regimes share one geometry kernel. Undifferenced residuals are computed once; the strategy selected by `differencingModeFor` then applies no transform (PPP), a between-satellite single difference against a pivot (PPP-RTK), or a between-receiver → between-satellite double difference (baseline) before the common Kalman update.

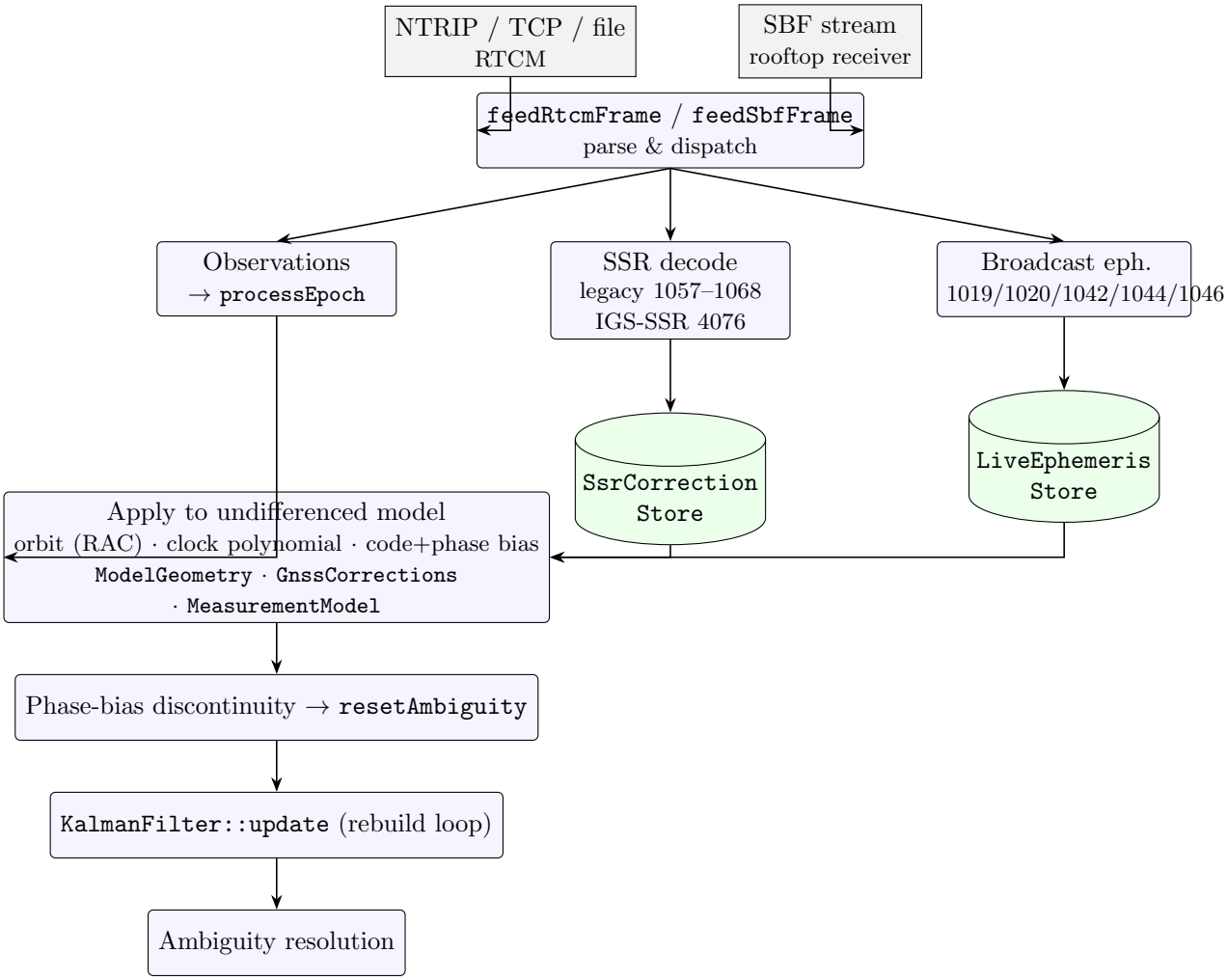


Figure 3: Real-time SSR correction pipeline. Observations, SSR corrections, and broadcast ephemerides are decoded from the byte stream into two stores; the undifferenced observation model then overlays the precise orbit (RAC frame), clock polynomial, and code/phase biases before the Kalman update. A phase-bias discontinuity counter forces an ambiguity reset ahead of the update. See `RealTimeEngine::processEpoch`.

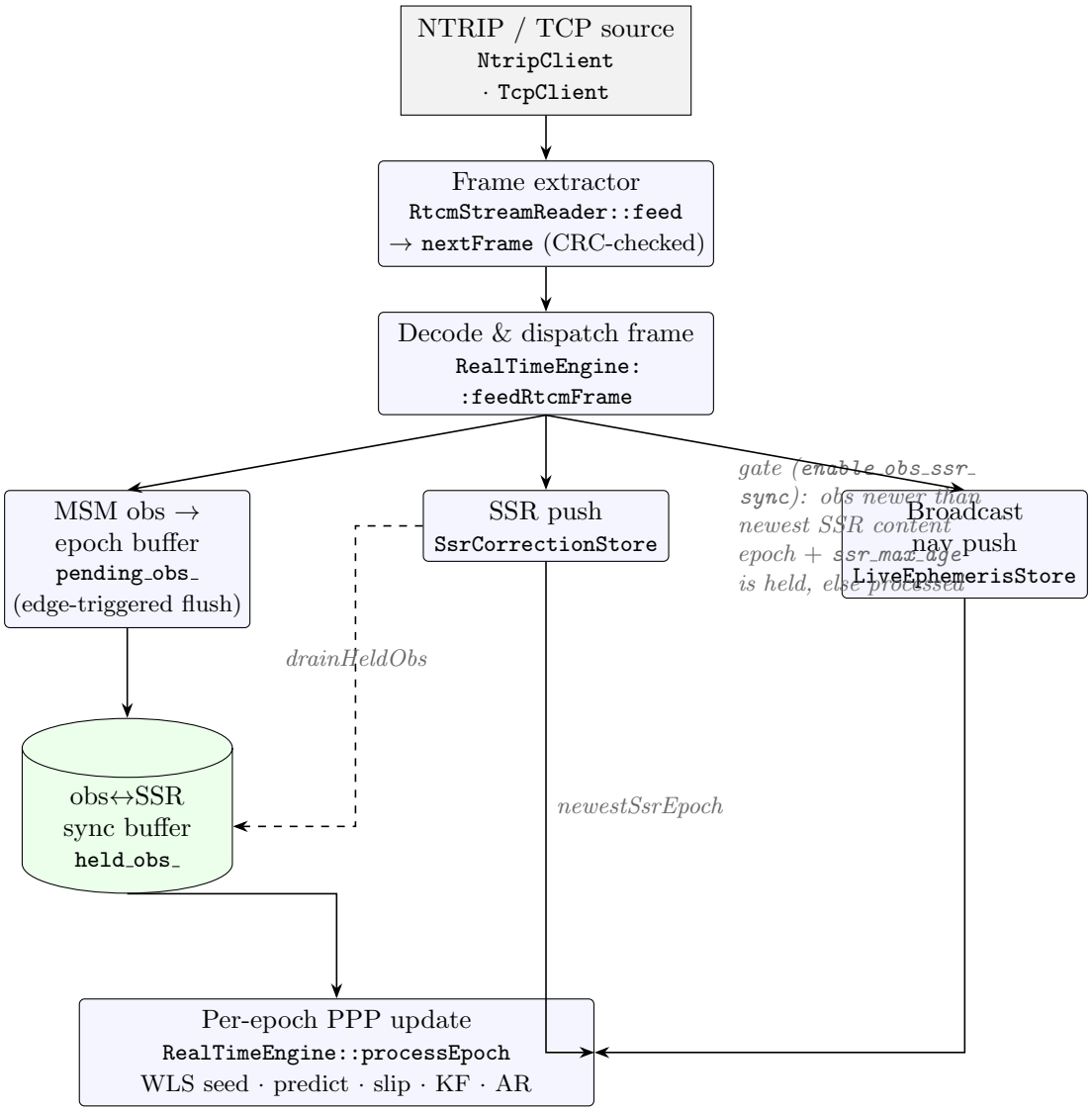


Figure 4: Real-time streaming front-end ahead of the SSR correction stage of Figure 3. Raw bytes from `NtripClient/TcpClient` are reassembled into CRC-validated RTCM3 frames by `RtcmStreamReader`, then decoded and dispatched by `feedRtcmFrame`: MSM observations accumulate in an edge-triggered epoch buffer, while SSR corrections and broadcast ephemerides flow into their stores. When `enable_obs_ssr_sync` is set, an obs epoch that runs ahead of the SSR coverage horizon (newest SSR content epoch + `ssr_max_age`) is held in `held_obs_` and re-attempted by `drainHeldObs` once an SSR push advances the horizon; otherwise it is passed straight to `processEpoch`. See the real-time engine.

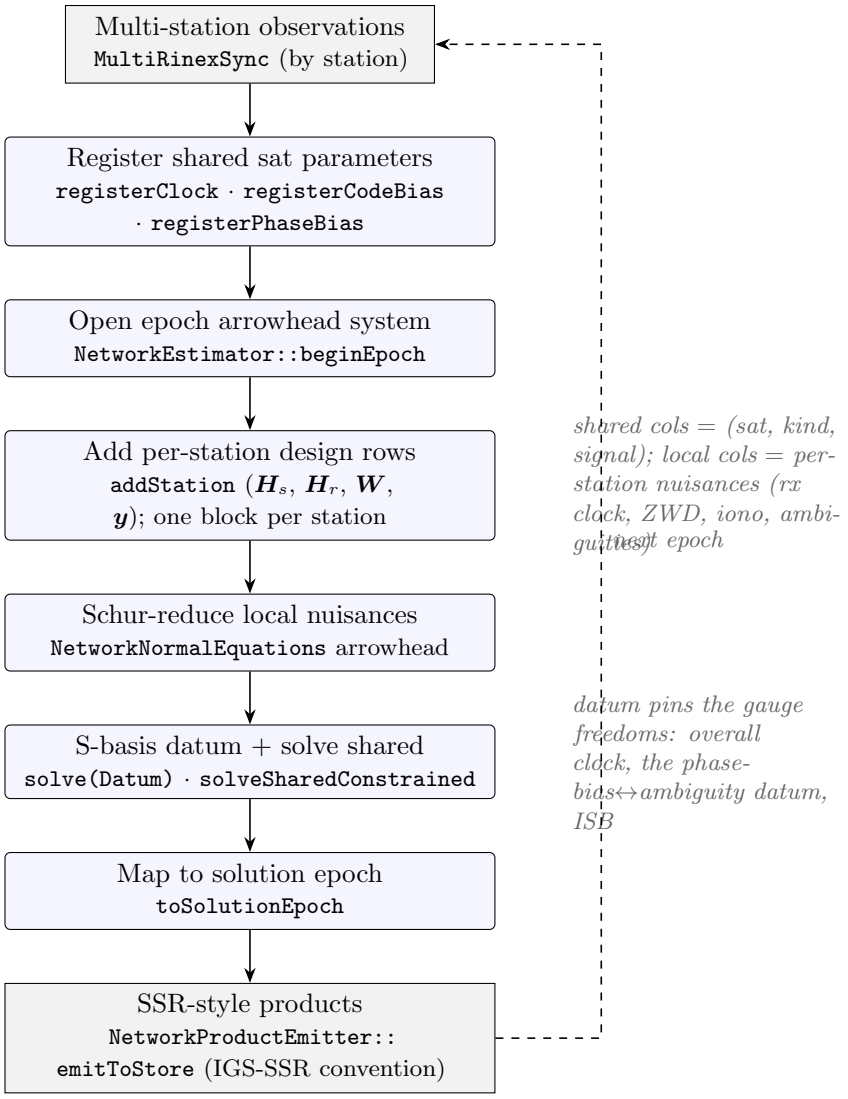


Figure 5: PPP-RTK network producer joint estimator (`NetworkEstimator`). Each epoch the shared satellite-side columns—one per (satellite, parameter-kind, signal) for the estimable clock, code bias, and phase bias—are re-registered at the top of every epoch (`reset`, then `registerClock` / `registerCodeBias` / `registerPhaseBias`, plus `registerVtecCoeff` for the network-wide SH-VTEC coefficients when `network.estimate_iono_sh` is on), after which `beginEpoch` opens an arrowhead normal-equation system of that width. `addStation` contributes one pre-formed block of local-vs-shared design rows per station (the network applies no receiver-side differencing); the local nuisances are Schur-reduced out, leaving a reduced shared system that is rank-deficient by the gauge freedoms. `solve` pins those with an explicit minimum-constraint S-basis datum on a copy of the reduced system, and `toSolutionEpoch` maps the shared estimate to an emittable record that `NetworkProductEmitter` serializes in IGS-SSR convention. The `Application::runNetwork` driver runs this core in a live per-epoch loop over the synced `MultiRinexSync` stations, but is itself only orchestration: the two-pass station assembly (`networkPassAHead_`/`networkPassB_`, `app/ApplicationNetworkAssembly`) feeds `addStation`, the datum and solve run in `networkSolveGlue`, and the emit step in `networkEmitStore` — which, when `network.closed_loop.enable` is armed, additionally drives a held-out PPP-RTK consumer against the freshly emitted products, a between-satellite single-difference (BSSD) consumer by default or an undifferenced one under `closed_loop.consumer_undifferenced`. See the network mode and its producer-architecture section.